



SISTEMAS OPERATIVOS - TEORÍA
12 de Junio de 2026

Nombre _____ DNI _____
Apellidos _____ Grupo _____

Cuestión 1 (2 puntos). Considere el siguiente código de un programa *randf* en un sistema POSIX.

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
int main(int argc, char **argv) {
    int fd1, fd2, bytes, bread = 0;
    char *endptr;
    if (argc < 3 || (bytes = strtol(argv[2], &endptr, 10)) <= 0 || *endptr != '\0') {
        fprintf(stderr, "usage: %s name bytes\n", argv[0]);
        exit(EXIT_FAILURE);
    }
    fd1 = open(argv[1], O_CREAT | O_TRUNC | O_WRONLY, S_IRUSR | S_IWUSR);
    fd2 = open("/dev/urandom", O_RDONLY);
    if (fd1 < 0 || fd2 < 0) {
        perror("open");
        exit(EXIT_FAILURE);
    }
    while (bread < bytes) {
        char c;
        if (read(fd2, &c, 1) <= 0 || write(fd1, &c, 1) <= 0)
            break;
        bread++;
    }
    close(fd1);
    close(fd2);
    return 0;
}
```

Responde **razonadamente** a las siguientes cuestiones:

- Explica qué hace el programa si lo invocamos como `./randf fichero1 2032`. Justifica el papel de los argumentos usados en las llamadas a `open` del código anterior.
- Supón que el programa se ejecuta en un sistema de ficheros de bloques indexados (tipo Unix), con un tamaño de bloque de 2048 bytes, en el que se usan identificadores de bloque de 32 bits y nodos-i estándar (12 entradas directas y hasta tres niveles de indirección). Si invoco `./randf fichero2 1080000`, ¿cuántos bloques necesitaría reservar el sistema de ficheros para fichero2?
- ¿Qué estructuras del sistema de ficheros se verían modificadas o actualizadas por la ejecución de los siguientes comandos (describe y justifica los cambios), uno detrás de otro? Supóngase que todos los comandos se ejecutan con éxito, es decir no producen ningún error.
 - `mkdir dir1`
 - `echo "hola mundo" > dir1/fichero4`
 - `ln dir1/fichero4 fichero5`
 - `ln -s dir1/fichero4 fichero6`
 - `rm -rf dir1`
- Explica qué pasaría si, tras la ejecución de los comandos anteriores, se ejecutase el comando `cat` sobre los ficheros `fichero5` y `fichero6`.

Cuestión 2 (2 puntos). Considere un sistema operativo con un planificador de procesos basado en Colas Multinivel con Realimentación (MLFQ) diseñado para penalizar procesos intensivos en CPU y favorecer a los interactivos, pero con un mecanismo de seguridad para evitar la inanición. El sistema gestiona tres niveles de prioridad (Q_0 alta, Q_1 media, Q_2 baja), donde los nuevos procesos ingresan siempre en Q_0 . Las políticas de planificación son Round Robin con $q=2$ para Q_0 , Round Robin con $q=4$ para Q_1 y FCFS para Q_2 , existiendo expropiación inmediata si llega un proceso a una cola más prioritaria que la actual. Un proceso desciende un nivel sólo si agota su cuanto completo sin bloquearse. Si cede la CPU por E/S, incluso al terminar el cuanto, mantiene su prioridad al regresar (encolándose al final de su cola correspondiente). Finalmente, se aplica una política de envejecimiento (aging) por la cual si un proceso permanece en Q_2 durante 6 unidades de tiempo consecutivas sin bloquearse, es ascendido inmediatamente al final de Q_1 . En caso de coincidencia en una misma cola, el proceso que ya estaba listo tiene preferencia sobre el que acaba de regresar de una E/S. Considere los siguientes procesos (tiempos en unidades de tiempo, u.t.):

Proceso	Llegada	Ráfagas (CPU, E/S, CPU)
A	0	5, 2, 6
B	1	3, 4, 2
C	3	12
D	9	2, 1, 2

Se pide:

- a) **(1,5 puntos)** Rellene el cronograma de ejecución. Marque con X_i cuando el proceso esté usando la CPU y provenga de la cola Q_i . Marque con B cuando esté Bloqueado por E/S, y ubíquelo en la cola correspondiente cuando esté Listo.

	00	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16
A:5,2,6																	
B:3,4,2																	
C:12																	
D:2,1,2																	
RR2																	
RR4																	
FCFS																	
	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33
A:5,2,6																	
B:3,4,2																	
C:12																	
D:2,1,2																	
RR2																	
RR4																	
FCFS																	

b) **(0,25 puntos)** Calcule el tiempo de respuesta de los cuatro procesos.

c) **(0,25 puntos)** Justifique si en algún instante se ha producido algún evento de envejecimiento (aging) y, en caso afirmativo, qué procesos se han visto afectados.

Cuestión 3 (2 puntos). Un programador poco experimentado desarrolla el programa C para Linux que se muestra más abajo. El objetivo que se persigue con el programa es crear un fichero "log.txt" en el que el proceso hijo y el principal (padre) escriban únicamente las siguientes líneas de texto en el fichero (suponiendo que tienen PIDs 12345 y 12344, respectivamente):

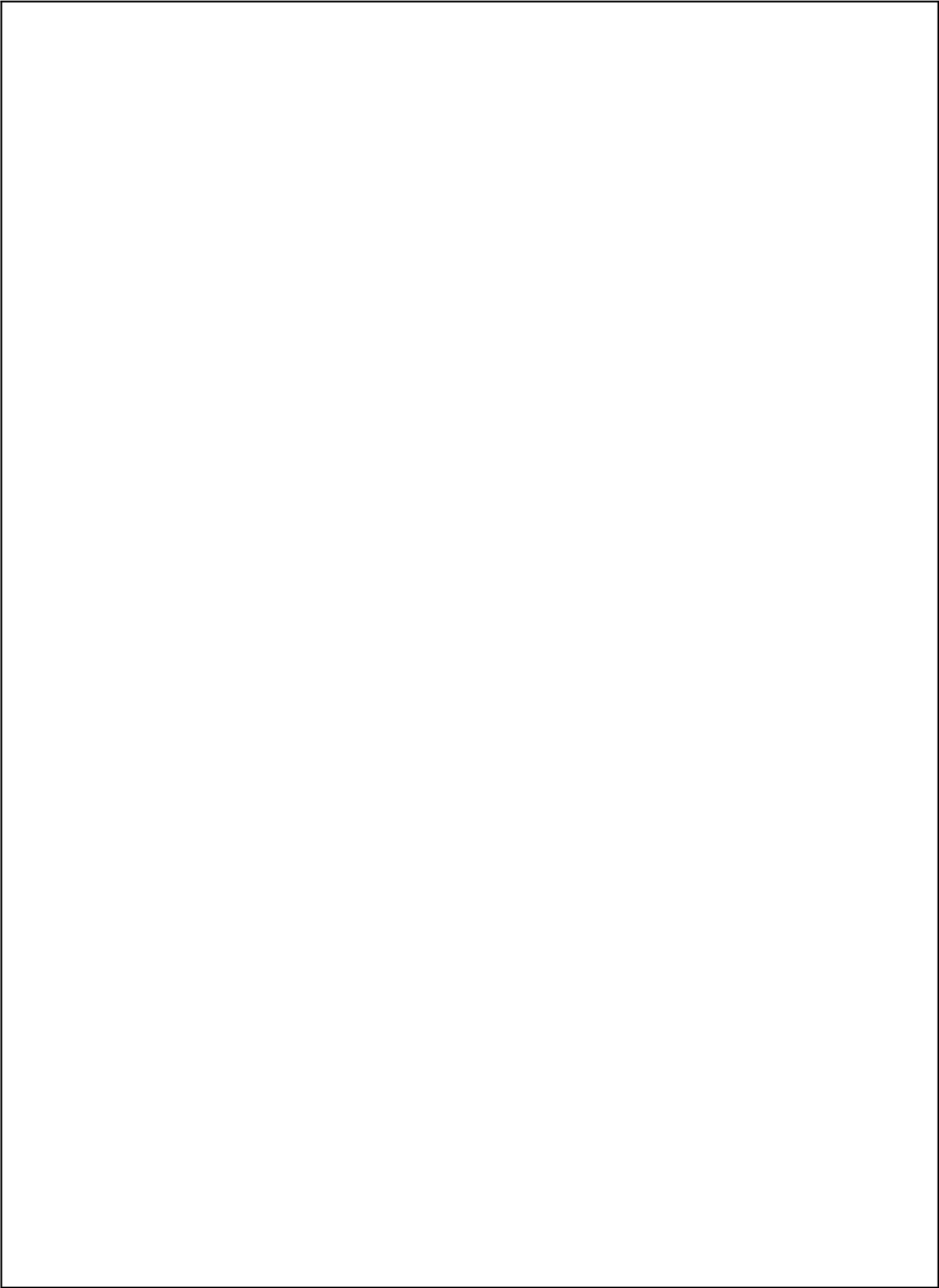
I am the child and my PID is 12345

I am the parent and my PID is 12344. Child exit status: 0

Asimismo el hijo ejecutará el comando `env` que escribe por pantalla la tabla de variables de entorno del proceso.

<pre> #include <unistd.h> // .. Resto de includes... void sigterm_handler(int sig) { const char msg[] = "SIGTERM RECEIVED\n"; write(STDOUT_FILENO, msg, sizeof(msg) - 1); } int main(void) { int fd; pid_t pid, childpid; char buffer[128]; int len; fd = open("log.txt", O_WRONLY O_CREAT O_TRUNC, 0644); pid = fork(); if (pid == 0) { close(fd); fd = open("log.txt", O_WRONLY O_CREAT O_TRUNC, 0644); len = snprintf(buffer, sizeof(buffer), "I am the child and my PID is %d\n", getpid()); } </pre>	<pre> write(fd, buffer, len); close(fd); execl("env"); exit(1); } else { struct sigaction sa; int status; sa.sa_handler = sigterm_handler; sigemptyset(&sa.sa_mask); sa.sa_flags = 0; sigaction(SIGTERM, &sa, NULL); status = wait(&childpid); len = snprintf(buffer, sizeof(buffer), "I am the parent and my PID is %d\nChild exit status:%d\n", getpid()); write(fd, buffer, len); close(fd); } return 0; } </pre>
---	--

- (0,75 puntos)** Lamentablemente, el programa anterior tiene tres errores en la implementación (cada error puede abarcar una o varias sentencias consecutivas). Identifique los errores, justificando razonadamente por qué se consideran errores. **Nota:** Se debe asumir que las llamadas `fork()`, `write()`, `sigaction()` y `open()` funcionan correctamente, es decir, no retornan -1 cuando el programa se ejecuta.
- (0,75 puntos)** Indique cómo subsanar los 3 errores identificados, proponiendo modificaciones que solo impliquen la eliminación de líneas de código y/o el reemplazo de parámetros y valores de retorno en llamadas del código suministrado.
- (0,5 puntos)** Considerando el código ya corregido, suponga que se envía la señal SIGTERM al proceso padre mientras el programa está en ejecución y el proceso hijo está ya creado. En este supuesto, se desea garantizar que el proceso padre imprima su mensaje en el fichero solamente cuando el hijo haya finalizado su ejecución. ¿Es necesario realizar alguna modificación adicional en el código para garantizar esto? Razone la respuesta, indicando en su caso qué modificaciones serían necesarias.



Cuestión 4 (2 puntos). Se desea implementar un nuevo mecanismo de sincronización llamado hemiforo. Se trata de un objeto que tiene un atributo entero sin signo, **value**, y una operación **H**, que toma dos argumentos **limit** y **delta**, y que se comporta del siguiente modo:

- Mientras **value** sea inferior a **limit**, el hilo que invoca la llamada a **H** queda bloqueado.
- Si **value** es mayor o igual que **limit**, a **value** se le suma el valor de **delta** de forma atómica, y si **delta** es positivo se desbloquea a todos los hilos bloqueados en el hemiforo para que comprueben si pueden proseguir (si **value** es mayor o igual al valor de **limit** en la llamada a **H** que los dejó bloqueados).

Se pide:

- a) **(1 punto)** Complete el código siguiente utilizando mutex y variables de condición estándar de C (o semáforos POSIX, si se desea) para implementar hemiforos.

<pre>#include <pthread.h> typedef struct hemiforo { unsigned value; // COMPLETAR } hemiforo_t; /* Inicialización del hemiforo */ void hemiforo_init(hemiforo_t *h, unsigned init_val) { // COMPLETAR }</pre>	<pre>/* Destrucción */ void hemiforo_destroy(hemiforo_t *h) { // COMPLETAR } /* Operación H */ void hemiforo_H(hemiforo_t *h, unsigned limit, int delta) { // COMPLETAR }</pre>
---	--

- b) **(1 punto)** Indica cómo puede utilizarse un hemiforo para implementar un semáforo.

<pre>typedef struct semaphore { hemiforo_t h; } semaphore_t; /* Inicialización del semáforo */ void semaphore_init(semaphore_t *s, unsigned value) { // COMPLETAR } /* Operación V / post del semáforo */ void semaphore_post(semaphore_t *s) { // COMPLETAR }</pre>	<pre>/* Destrucción del semáforo */ void semaphore_destroy(semaphore_t *s) { // COMPLETAR } /* Operación P / wait del semáforo */ void semaphore_wait(semaphore_t *s) { // COMPLETAR }</pre>
--	---

Cuestión 5 (2 puntos) Un sistema utiliza gestión de memoria virtual con paginación bajo demanda, donde las páginas tienen un tamaño de 4 KiB, direcciones de memoria de 48 bits y una memoria principal de 8 GB. Se ejecuta el siguiente programa en dicho sistema:

```
C/C++
#include <stdio.h>
#include <stdlib.h>
#include <sys/mman.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

#define N 256
int *shared_data;

int main(int argc, char *argv[]) {
    int fd, fo;
    pid_t pid = 0;

    //A
    shared_data = mmap(NULL, N * sizeof(int), PROT_READ | PROT_WRITE, MAP_SHARED |
MAP_ANONYMOUS, -1, 0);
    if (shared_data == MAP_FAILED) {
        perror("mmap");
        exit(1);
    }

    //B
    pid = fork();

    //C
    if (pid == 0) {
        for (int i = 0; i < N; i++) {
            shared_data[i] = i * 2;
        }
        execlp("echo", "echo", "Shared data is modified", (char *)NULL);
        perror("exec");
        exit(1);
    } else {
        wait(NULL);

        for (int i = 0; i < N; i++) {
            printf("shared_data[%d] = %d\n", i, shared_data[i]);
        }
    }
    // D
    munmap(shared_data, N * sizeof(int));
    exit(0);
}
```

Tome como punto de partida la salida del mapa de memoria entre A y B.

```
cat /proc/1509/maps
59b99806a000-59b99806b000 r--p 00000000 00:43 4222124652544741 /workspaces/mem_2
59b99806b000-59b99806c000 r-xp 00001000 00:43 4222124652544741 /workspaces/mem_2
59b99806c000-59b99806d000 r--p 00002000 00:43 4222124652544741 /workspaces/mem_2
59b99806d000-59b99806e000 r--p 00002000 00:43 4222124652544741 /workspaces/mem_2
59b99806e000-59b99806f000 rw-p 00003000 00:43 4222124652544741 /workspaces/mem_2
59b9b1bf6000-59b9b1c17000 rw-p 00000000 00:00 0 [heap]
7dbb7937e000-7dbb79381000 rw-p 00000000 00:00 0
```

```

7dbb79381000-7dbb793a3000 r--p 00000000 08:40 45541
/lib/x86_64-linux-gnu/libc-2.31.so
7dbb793a3000-7dbb794fc000 r-xp 00022000 08:40 45541
/lib/x86_64-linux-gnu/libc-2.31.so
7dbb794fc000-7dbb7954b000 r--p 0017b000 08:40 45541
/lib/x86_64-linux-gnu/libc-2.31.so
7dbb7954b000-7dbb7954f000 r--p 001c9000 08:40 45541
/lib/x86_64-linux-gnu/libc-2.31.so
7dbb7954f000-7dbb79551000 rw-p 001cd000 08:40 45541
/lib/x86_64-linux-gnu/libc-2.31.so
7dbb79551000-7dbb79555000 rw-p 00000000 00:00 0
7dbb79555000-7dbb7955b000 r--p 00000000 08:40 45584
/lib/x86_64-linux-gnu/libpthread-2.31.so
7dbb7955b000-7dbb7956b000 r-xp 00006000 08:40 45584
/lib/x86_64-linux-gnu/libpthread-2.31.so
7dbb7956b000-7dbb79571000 r--p 00016000 08:40 45584
/lib/x86_64-linux-gnu/libpthread-2.31.so
7dbb79571000-7dbb79572000 r--p 0001b000 08:40 45584
/lib/x86_64-linux-gnu/libpthread-2.31.so
7dbb79572000-7dbb79573000 rw-p 0001c000 08:40 45584
/lib/x86_64-linux-gnu/libpthread-2.31.so
7dbb79573000-7dbb79579000 rw-p 00000000 00:00 0
7dbb7957f000-7dbb79580000 r--p 00000000 08:40 45534
/lib/x86_64-linux-gnu/ld-2.31.so
7dbb79580000-7dbb795a0000 r-xp 00001000 08:40 45534
/lib/x86_64-linux-gnu/ld-2.31.so
7dbb795a0000-7dbb795a8000 r--p 00021000 08:40 45534
/lib/x86_64-linux-gnu/ld-2.31.so
7dbb795a8000-7dbb795a9000 rw-s 00000000 00:01 2049 /dev/zero
7dbb795a9000-7dbb795aa000 r--p 00029000 08:40 45534
/lib/x86_64-linux-gnu/ld-2.31.so
7dbb795aa000-7dbb795ab000 rw-p 0002a000 08:40 45534
/lib/x86_64-linux-gnu/ld-2.31.so
7dbb795ab000-7dbb795ac000 rw-p 00000000 00:00 0
7ffc8330a000-7ffc8332b000 rw-p 00000000 00:00 0 [stack]
7ffc8339e000-7ffc833a2000 r--p 00000000 00:00 0 [vvar]

```

- a) **(1 punto)** Identifique las regiones lógicas de memoria (y cuántas páginas ocupan) en los comentarios A, B, C y D, tomando como referencia el mapa de memoria entre A y B.
- b) **(1 punto)** Determine cuántos fallos de página se producen al pasar del punto A al B y del punto C al punto D.